Python-Based Malware Simulator

Cybersecurity Activity Report

Student Name: Dharmik Mehta
Subject: Cybersecurity
Institution: GMIU

1. Objective

The objective of this activity is to develop a Python-based malware simulator that demonstrates the basic behavior of viruses, worms, and Trojans in a safe and controlled environment. The activity helps students understand malware propagation, disguise techniques, potential security impacts, and preventive measures.

2. Introduction

Malware is malicious software designed to damage computer systems, disrupt operations, steal information, or gain unauthorized access. Different malware types use different methods to affect their targets.

This project uses Python to simulate three common types of malware. All demonstrations operate on virtual data within the program. No real files are infected, no network connections are made, and no malicious actions are performed.

3. Types of Malware Simulated

A. Virus

A computer virus typically attaches itself to a legitimate file or program and may spread when the infected content is executed or shared.

In this simulation, a virtual collection of files is created. The program marks eligible text and Python files as affected, illustrating the concept of file-based infection.

Potential impact: File corruption, data loss, application disruption, and further malware propagation.

B. Worm

A worm is malware capable of self-propagation between computer systems, often through networks or security vulnerabilities.

In this simulation, five virtual computers are represented using a Python dictionary. The program marks each virtual computer as affected to demonstrate propagation.

Potential impact: Network congestion, rapid spread, system performance degradation, and disruption of services.

C. Trojan Horse

A Trojan is malicious software disguised as a legitimate or useful application. Unlike a conventional virus or worm, it relies on deception to persuade a user to run it.

In this simulation, a fictional game installer is displayed as apparently legitimate software. The program then prints a message representing the detection of simulated hidden behavior.

Potential impact: Information theft, unauthorized access, installation of additional malware, and compromise of sensitive information.

4. Tools and Technologies

- Programming language: Python 3
- Development environment: IDLE, VS Code, or PyCharm
- Libraries: Python standard library only
- Interface: Command-line terminal

5. Methodology

1. Create a Python program containing separate functions for the three malware types.
2. Represent files and computers using virtual data structures.
3. Simulate file-based effects for the virus.
4. Simulate propagation among virtual computers for the worm.
5. Demonstrate a trusted-looking application and hidden behavior for the Trojan.
6. Display the results and a summary of security recommendations.
7. Capture a screenshot of the executed program for submission.

6. Results and Analysis

The virus simulation marks three of four virtual files as affected. This demonstrates how file-associated malware can affect selected files.

The worm simulation marks all five virtual computers as affected. This demonstrates how malware propagation can extend beyond an initial system.

The Trojan simulation shows how an application can appear legitimate while concealing suspicious behavior. It highlights the importance of verifying software sources.

These results are illustrative outputs of a controlled simulation. They do not measure the performance or behavior of real malware.

7. Preventive Measures

- Install reputable antivirus or endpoint protection software.
- Keep operating systems and applications updated.
- Download software only from trusted sources.
- Avoid suspicious links, attachments, and unexpected downloads.
- Use firewalls and appropriate access permissions.

- Maintain regular backups of important data.
- Provide cybersecurity awareness training to users.

8. Safety Considerations

The simulator performs no real infection, replication, data theft, persistence, or network propagation. All simulated activity is limited to in-memory data structures and printed messages. The program is intended solely for educational purposes.

9. Conclusion

The Python-based malware simulator demonstrates the fundamental differences between viruses, worms, and Trojans. It illustrates file-based infection, virtual system propagation, and software disguise in a safe environment. This activity improves understanding of malware behavior and emphasizes the importance of cybersecurity awareness and preventive measures.

10. Submission Items

Output screenshot:-

